Zadanie č. 1: Neurónové siete **- Dokumentácia**

Meno:Martin Kasznár **Krúžok:piatok od 10:00 do 12:00**

Úvod

Zadanie súvisí s tým, že je nám predstavený dataset o dĺžke 13140 riadkov s 20 stĺpcami s rôznymi typmi hodnôt a jedným cieľovým stĺpcom s hodnotou áno/nie, pričom tých 20 stĺpcov reprezentuje 13140 rôznych klientov, na ktorých sú merané metriky a cieľový stĺpec predstavuje či sa nakoniec rozhodli pre termínovaný vklad. Ide o to spraviť merací nástroj či by sa nevidený klient, na ktorého by sme mali aspoň necieľové stĺpce (no nie tie výrazne ovplyvňujúce cieľovú premennú ako napr. duration-dĺžka posledného kontaktu) pravdepodobne rozhodol, že chce termínovaný vklad či, že ho nechce.
K tomu máme pristúpiť tak, že tento dataset ešte treba vyčistiť a potom naň treba po 1.) **zostaviť a predvádzať jednoduchší klasifikačný model z knižnice sklearn,** po 2.) **predvádzať EDA na neškálovaných a stratovo nezmanipulovaných dátach,** po 3.) **zostaviť a predvádzať plnohodnotnú neurónovú sieť.**

0.0 edamap() - zľahčenie manipulácie s datasetom

```
NOTHING = 'nic'
EDA_MAPPINGS = {
    'age':            {},                  #NUMERICAL
    'job':            {'unknown': NOTHING, 'admin.': 0, 'blue-collar': 1, 'entrepreneur': 2, 'housemaid': 3, #categ
                       'management': 4, 'retired': 5, 'self-employed': 6, 'services': 7, 'student': 8,
                       'technician': 9, 'unemployed': 10},
    'marital':        {'unknown': NOTHING, 'divorced': 0, 'married': 1, 'single': 2},                        #categ
    'education':      {'unknown': NOTHING, np.nan: NOTHING, 'basic.4y': 0, 'basic.6y': 1, 'basic.9y': 2,     #categ
                       'high.school': 3, 'illiterate': 4, 'professional.course': 5, 'university.degree': 6},
    'default':        {'unknown': NOTHING, 'no': 0, 'yes': 1}, #binary
    'housing':        {'unknown': NOTHING, 'no': 0, 'yes': 1}, #binary
    'loan':           {'unknown': NOTHING, 'no': 0, 'yes': 1}, #binary
    'contact':        {'cellular': 0, 'telephone': 1},         #binary
    'month':          {np.nan: NOTHING, 'jan': 0, 'feb': 1, 'mar': 2, 'apr': 3, 'may': 4, 'jun': 5,          #categ
                       'jul': 6, 'aug': 7, 'sep': 8, 'oct': 9, 'nov': 10, 'dec': 11},
    'day_of_week':    {np.nan: NOTHING, 'mon': 0, 'tue': 1, 'wed': 2, 'thu': 3, 'fri': 4},                   #categ
    'duration':       {np.nan: NOTHING}, #NUMERICAL
    'campaign':       {np.nan: NOTHING}, #NUMERICAL
    'pdays':          {np.nan: NOTHING}, #NUMERICAL
    'previous':       {np.nan: NOTHING}, #NUMERICAL
    'poutcome':       {'failure': 0, 'nonexistent': 1, 'success': 2},                                         #categ
    'emp.var.rate':   {np.nan: NOTHING}, #NUMERICAL
    'cons.price.idx': {np.nan: NOTHING}, #NUMERICAL
    'cons.conf.idx':  {},                #NUMERICAL
    'euribor3m':      {np.nan: NOTHING}, #NUMERICAL
    'nr.employed':    {np.nan: NOTHING}, #NUMERICAL
    'subscribed':     {'no': 0, 'yes': 1}                    #binary
}
```

Pre ľahkosť navigácie množstva dát aj bez upozornenia a hlbšie v práci budú drvivú väčšinu času dáta namapované nasledovne: unknown a prázdne miesto->NOTHING string konštanta, slová->čísla, plne automatizovane funkciou **edamap().**

1.2 cullnull() - mazanie nullov a undefined

– Odstráňte stĺpce, ktoré sa nedajú použiť pri ďalšom spracovaní a *null* resp. *undefined* hodnoty (pozor na odstraňovanie *null* resp. *undefined* hodnôt - aby ste neprišli o príliš veľa dát, ak v stĺpci chýba príliš veľa hodnôt, zvážte, či nie je rozumnejšie odstrániť celý stĺpec). **0.5b**

Po prezretí tabuľky v Exceli sa zistilo, že sa tu nachádza jedna hodnota nedeklarovaná v popise stĺpcov a tou je **prázdne miesto**. Sú tu ale aj iné hodnoty z popisu stĺpcov, ktoré by sa zrejme zle modelovali ako **string unknown** a tak ich vedieme tiež ako NOTHING string konštantu. Keďže nemajú priamu informačnú hodnotu o niečom, čo sa uskutočnilo v živote človeka alebo o tom aký človek je, je 1. riešenie mazať riadky čo majú prázdne miesto, no to by nemuselo prejsť, ak by sme tým zmazali polovicu či 2/3 datasetu a tak sa model nenaučí nič. Padlo rozhodnutie funkciou **cullnull(hranica iného typu mazania)**, ak sa NOTHING vyskytuje pod stĺpcom málokrát mazať riadky, ak moc veľakrát, rovno odstrániť celý stĺpec a riadkami nemanipulovať. Takto sú dáta snáď výpovednejšie a hlavne o krok bližšie k úplnej numerizácii.

```
>>> df.eq('unknown').sum()
...
age                0
job              116
marital           29
education        594
default         2412
housing          310
loan             310
contact            0
month              0
day_of_week        0
duration           0
campaign           0
pdays              0
previous           0
poutcome           0
emp.var.rate       0
cons.price.idx     0
cons.conf.idx      0
euribor3m          0
nr.employed        0
subscribed         0
dtype: int64
>>>
```

```
>>> df.eq('unknown').sum().sum()
...
0
>>> df.isna().sum().sum()
...
0
>>> df.eq('nic').sum().sum()
...
0
>>>
```

-string unknown->'nic'<-prázdne miesto-v

```
>>> df.isna().sum()
age                0
job                0
marital            0
education          5
default            0
housing            0
loan               0
contact            0
month              5
day_of_week        5
campaign           5
pdays              5
previous           5
poutcome           0
emp.var.rate       5
cons.price.idx     5
cons.conf.idx      0
euribor3m       6543
nr.employed        5
subscribed         0
dtype: int64
>>>
```

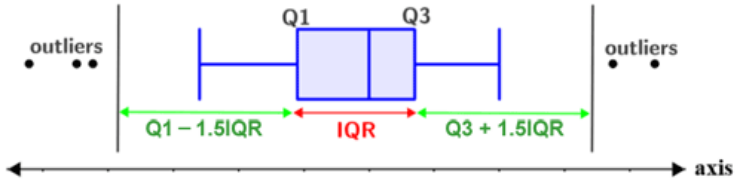
```
0.0 Eda mapped : unknown and np.nan-> nic , words->numbers
1.2 Culled nulls under job : 116x nic rows removed
1.2 Culled nulls under marital : 23x nic rows removed
1.2 Culled nulls under education : 557x nic rows removed
1.2 Culled nulls under default : (ENTIRE COL DELETED) cuz has 2222x nic rows
1.2 Culled nulls under housing : 297x nic rows removed
1.2 Culled nulls under month : 5x nic rows removed
1.2 Culled nulls under day_of_week : 3x nic rows removed
1.2 Culled nulls under duration : 5x nic rows removed
1.2 Culled nulls under campaign : 5x nic rows removed
1.2 Culled nulls under pdays : 5x nic rows removed
1.2 Culled nulls under previous : 4x nic rows removed
1.2 Culled nulls under emp.var.rate : 5x nic rows removed
1.2 Culled nulls under cons.price.idx : 5x nic rows removed
1.2 Culled nulls under euribor3m : (ENTIRE COL DELETED) cuz has 6006x nic rows
1.2 Culled nulls under nr.employed : 5x nic rows removed
OK
>>>
```

edamap()+cullnull(2000)

1.1 culloutl() - mazanie outlierov

– Prezrite si stĺpce v databáze (popis stĺpcov na konci tohto zadania). Podľa popisu zistite, či sa v jednotlivých stĺpcoch nachádzajú outliery (neobvyklé hodnoty) a odstráňte ich. **0.5b**

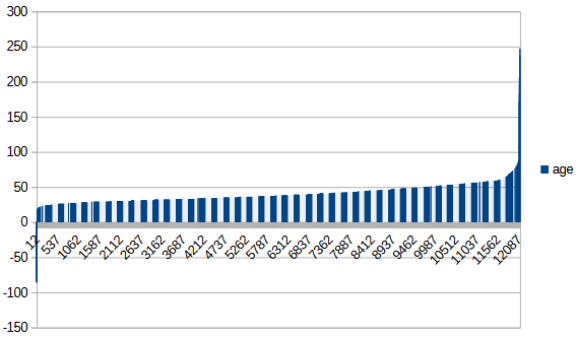
V prvom rade si vymenujme typy stĺpcov: máme numerické, kategorické a binárne aspoň pre túto tabuľku. V kategorických nejde zas o také rozsahy, keďže neskôr ich v zadaní mienime zmeniť na onehot encoding, čo je vlastne explózia jednej kategórie na mnoho s binárnym tvrdením či je to jedna z podkategórií kategórie. Cez toto by mohla v neurónovej sieti viesť jednoduchá čiara rovnako ako cez binárne kategórie, ale v **numerických stĺpcoch** by nám mohlo zdvihnúť rád požadovanej krivky náhle nezmyselne záporne či kladne veľké číslo. Existuje princíp IQR outlier mazania, kde i o to, že v **culloutl(pozícia ľavého percentilu, pozícia pravého percentilu)** hľadíme na to aké číslo máme na pozícii ľavého percentilu, to značíme Q1 a na pozícii pravého percentilu, to značíme Q3. Vypočítame IQR=Q3-Q1 a snahou je zabezpečiť aby všetko viac vľavo ako Q1-1.5IQR a viac vpravo ako Q3+1.5IQR šlo z dataframeu preč (riadok). Tento princíp je adekvátne vysvetlený na nižšom obrázku:

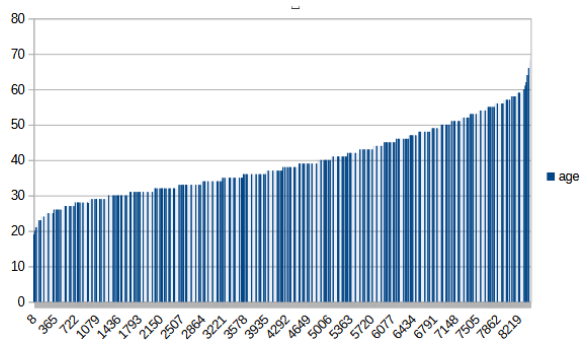


```
1.1 Culled outliers with IQR( 0.25 , 0.75 ): 3697 x rows removed
OK
>>>
```

edamap()+cullnull(2000)+culloutl(0.25, 0.75)

Demonštrujme prakticky:





Prevádzame na týchto stĺpcoch:

```
NUMERIC_COLS = ['age', 'duration', 'campaign', 'pdays', 'previous', 'emp.var.rate',
                'cons.price.idx', 'cons.conf.idx', 'euribor3m', 'nr.employed']
```

1.3 dummify() - onehot encoding

– Nečíselné stĺpce vhodne zakódujte. **0.5b**

V projekte nastáva problém **kategorických stĺpcov** a tu ide o to, že dočasné číselné mapovania napr. 4 a 5 si absolútne nie sú blízke v skladbe prianí klienta, kým pri age ako numerickom type stĺpca by tá skladba prianí mohla byť podobná. Toto sa stáva problémom, keď pod stĺpcom, pod ktorým môže byť viac možností nie sú zároveň ani dve protipólne vzájomne opačné. Sieť by to zvádzalo myslieť si, že blízkosť hodnôt znamená rovnaké prianie, ale napr. job 4 manažér a job 5 dôchodca nebude mať podobné tendencie. Podľa princípu onehot encoding explodujeme 0-10 na 11 rôznych stĺpcov s hodnotami 0 či 1 funkciou **dummify()**.

continent	labelEncoding	Africa	Asia	Europe	North America	Oceania	South America
Europe	2	0	0	1	0	0	0
Africa	0	1	0	0	0	0	0
Africa	0	1	0	0	0	0	0
Africa	0	1	0	0	0	0	0
South America	5	0	0	0	0	0	1

```
1.3 Performed dummification: ['job', 'marital', 'education', 'month', 'day_of_we
ek', 'poutcome'] dummified
OK
>>>
```

edamap()+cullnull(2000)+culloutl(0.25, 0.75)+dummify()

Demonštrujme (nová verzia programu stĺpce volá priamo podľa práce):

B
job
6
0
9
1
9
9
0
4
1

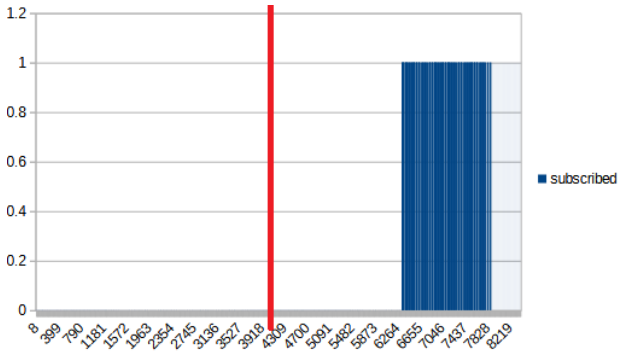
N	O	P	Q	R	S	T	U	V	W	X
job_0.0	job_1.0	job_2.0	job_3.0	job_4.0	job_5.0	job_6.0	job_7.0	job_8.0	job_9.0	job_10.0
0	0	0	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1	0
0	0	0	0	0	0	0	0	0	1	0
1	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	0

Prevádzame na týchto stĺpcoch:

```
CATEG_COLS = ['job', 'marital', 'education', 'month', 'day_of_week', 'poutcome']
```

1.35 upsamplebytarget() - nadvzorkovanie

```
>>> df.duplicated().sum()
2
```



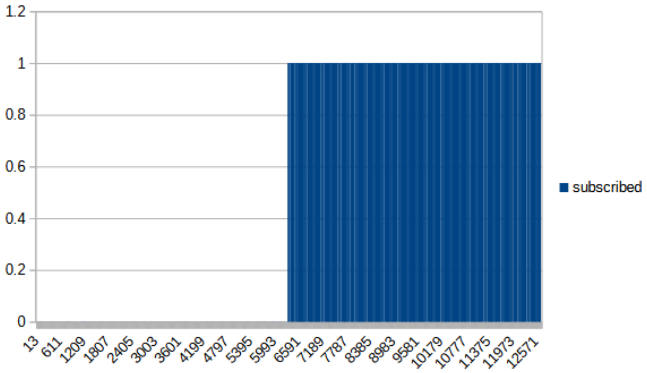
Už trochu spracovaný dataset (ale aj pred spracovaním) má ten problém, že podstatne viac ako polovica necieľových stĺpcov korešponduje s odmietnutím klientom termínovaného vkladu v cieľovom stĺpci. Sieť by to zvádzalo byť odmeňovanou za to, že odpovedá tak, aký je cieľ väčšiny datasetu namiesto čo najsprávnejšej odpovede na necieľové stĺpce. Na toto je vhodné použiť naivnejší upsampling vo funkcii **upsamplebytarget()**, ktorý sa pokúsi tomuto zabrániť duplikovaním riadkov, čo majú v cieľovom stĺpci minoritnú triedu (z grafu vidieť, že minoritou je rozhodne sa pre termínovaný vklad).

```
1.35 Upsampled by target: {0: 6358, 1: 2050} -> {0: 6358, 1: 6358}
OK
>>>
```

edamap()+cullnull(2000)+culloutl(0.25, 0.75)+dummify()+upsamplebytarget()

Demonštrujme:

```
>>> df.duplicated().sum()
4399
```



1.4 tvorba x/y množín, následné delenie na train/valid/test

– Vytvorte vstupné (X) a výstupné (y) dátové množiny. Vo vhodnom pomere rozdeľte dáta na trénovaciu, validačnú a testovaciu množinu. **0.5b**

x a **y** množina sa rodí tak, že prečítame tabuľku knižnicou pandas, čo ju nahodí do dataframeu df, potom na nej robíme čistenie dát (to je popísané vyššie), následne zostavíme hlbokú kópiu df oklieštěného o stĺpce duration a subscribed (subscribed zjavne lebo je cieľ, duration preto, lebo výrazne ovplyvňuje cieľovú premennú) preloženú do numpy.ndarray, toto nazývame **x** a hlbokú kópiu stĺpca df subscribed preloženú do numpy.ndarray, toto nazývame **y**. **Motivácia je naučiť model podľa x, čo máme na klienta, predpovedať y.**
Bežné pomery rozdelenia dát na trénovaciu:validačnú:testovaciu množinu bývajú 8:1:1, 7:1.5:1.5, 6:2:2, takže si vyberme napr. 8:1:1. tn, vd, tt nám slúžia na výber, stačí napísať za rovná sa 8,1,1 a toto programu cez train_test_split(…) povie, že trénovacej množine dajme 80% a 20% nám ostáva, následne z 20% ukrojí 50%, teda 10% testovacej množine a posledných 10% ide validačnej množine. **Motivácia delenia je v tom, že po epochách sa na trénovaciu množinu nahadzuje zlepšujúca sa krivka, ale aby nebola nevšeobecná na**

nevideného klienta ju uvoľňujeme validačnou množinou. Testovacia množina slúži na jediné otestovanie a toto má simulovať náhodného klienta, ktorý už v produkcii prišiel a chceme otestovať či bude mať podľa metrík pravdepodobne záujem o termínovaný vklad, lebo testovacia množina v trénovaní nie je videná modelom ani nepriamo cez tresty.

```
#-init-----------------------------------------
df = pd.read_csv('zadanie1-data-virgin.csv', sep=';')
pd.set_option('future.no_silent_downcasting', True)
silent=False
                ~~dataset cleaning~~
#1.4
x = copy.deepcopy(df.drop(columns=['duration','subscribed']).to_numpy())
y = copy.deepcopy(df['subscribed'].to_numpy())

tn, vd, tt = 8,1,1
x_train, x_temp, y_train, y_temp = train_test_split(
    x, y, test_size=(vd+tt)/(tn+vd+tt), random_state=42, stratify=y
)
x_val, x_test, y_val, y_test = train_test_split(
    x_temp, y_temp, test_size=(tt)/(vd+tt), random_state=42, stratify=y_temp
)
                ~~working with data~~
#-export-----------------
x=np.concatenate((x_train, x_val, x_test))
y=np.concatenate((y_train, y_val, y_test))
new_dfxfrag = pd.DataFrame(x, columns=df.drop(columns=['duration','subscribed']).columns)
new_dfyfrag = pd.Series(y, name='subscribed')
df = pd.concat([new_dfxfrag, new_dfyfrag], axis=1)
df.to_csv('out.csv', sep=';', index=False)
print('OK')
```

```
>>> len(x_train)
10172
>>> len(x_val)
1272
>>> len(x_test)
1272
>>> len(x)
12716
>>>
```

1.5 štandardizácia

– Dáta správne normalizujte alebo škálujte. **0.5b**

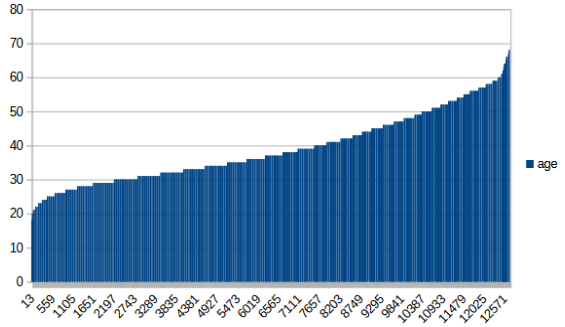
Pri dátach môže byť problém to, že by mali rôzne magnitúdy hoci by mali ekvivalentné tvary a na toto slúži štandardizovanie, ktoré nám zabezpečí, že keď už tvary nie sú plné najmä outlierov sa posunie dole aj magnitúda a to cez štandardizáciu, kde popisujeme dáta tým, že o koľko odlišné sú oproti strednej hodnote stĺpca. Správne štandardizovanie je to vtedy, keď **neleakneme** validačnú či testovaciu množinu trénovacej množine tak, že stredná hodnota aj smerodajná odchylka **nemajú** pochádzať aj z nich.

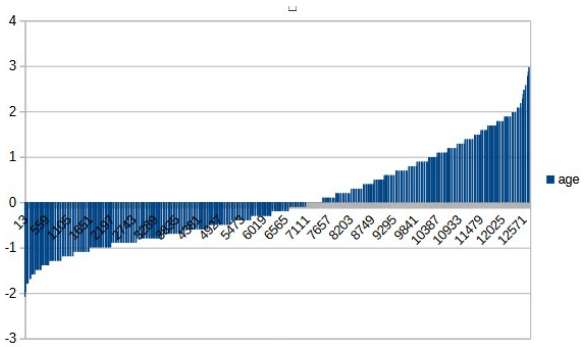
$$x_{norm} = \frac{x - \mu}{\sigma}$$

Bude to vplývať kladne na trénovanie neurónovej siete, keďže hyperparametre siete váhy a bias nebudú musieť byť menené tak často, keďže sú neuróny efektívnejšie využívané väčším množstvom prechádzajúcich informácií.

```
#1.5
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_val_scaled   = scaler.transform(x_val)
x_test_scaled  = scaler.transform(x_test)
```

Demonštrujme:





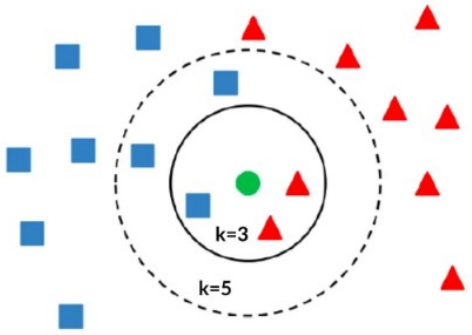
1.6 K-nearest neighbor

– Natrénujte jednoduchý klasifikačný model (tu Vám stačí Sklearn).**1b**

Podstata K-nearest neighbor spočíva v tom, že daný klient charakterizovaný všetkými metrikami je reprezentovaný **48-rozmerným bodom** v priestore a podľa toho či k=1, k=2, k=3 či viac, ak 3, tak si načíta 3 najbližších susedov cez metriku Euklidovskej vzdialenosti a následne sa rozhodne pre to, pre čo sa rozhodla väčšina týchto susedov.

$$d(x, y) = \sqrt{\sum_{i=1}^{n}(y_i - x_i)^2}$$

Trénovacia fáza by teda v programe spočívala v načítaní x_train,y_train, validačná v kontrole či takéto množstvo susedov dobre všeobecne popíše budúceho nikdy nevideného klienta a testovacia fáza je spomínaný jeden test akoby z produkcie.




```
k 20 | val_acc=0.743
k 21 | val_acc=0.742
k 22 | val_acc=0.745
k 23 | val_acc=0.744
k 24 | val_acc=0.741
k 25 | val_acc=0.738
k 26 | val_acc=0.733
k 27 | val_acc=0.733
k 28 | val_acc=0.728
k 29 | val_acc=0.733
k 30 | val_acc=0.733
k 31 | val_acc=0.733
Early stopping po 31 k-ckach (patience=30). Najlepšia val_acc=0.8836
Najlepšie k=1
Train accuracy: 99.52%
Test accuracy: 88.68%
OK
>>>
```

KNN pre bod v 2-rozmernom priestore |validácie->skoré zastavenie->oznámenie výhodného k a trénovacej a testovacej presnosti

Konštatujeme, že trénovacia presnosť dosahuje 99.52% a testovacia presnosť dosahuje 88.68%.

1.7 predvádzanie KNN s rôznymi rozdeleniami dátových množín

– Experimentujte s rozdelením dátových množín. Natrénujte model a vyhodnoťte na trénovacej **A** testovacej množine viackrát. Vytvorne **1** tabuľku s aspoň 4 experimentami s dosahnutými úspešnosťami. Zamyslite sa nad jednotlivými výsledkami a zhodnoťte, ako rozdelenie vplýva na celkovú úspešnosť.**1b**

rozdelenie dát	train úspešnosť	test úspešnosť
9:0.5:0.5	99.42%	88.68%
8:1:1	**99.52%**	**88.68%-vyhráva**
7:1.5:1.5	99.52%	87.11%
6:2:2	99.61%	85.22%
5:2.5:2.5	99.61%	82.57%

Vpravo v tabuľke vidíme, že pri pridelení čím väčšieho kusu datasetu trénovacej množine na úkor validačnej a testovacej sa samozrejme tým viac zvyšuje testovacia úspešnosť, keďže KNN pozná väčšie množstvo bodov po celom 48-rozmernom priestore, ktoré majú lepšiu výpovednú hodnotu pre klasifikáciu. V strede v tabuľke sa pravdepodobne nenachádza jednoduchý súvis s rozdelením dát, keďže rozdiely sú minimálne. Šedou je vyznačené bežné nastavenie.

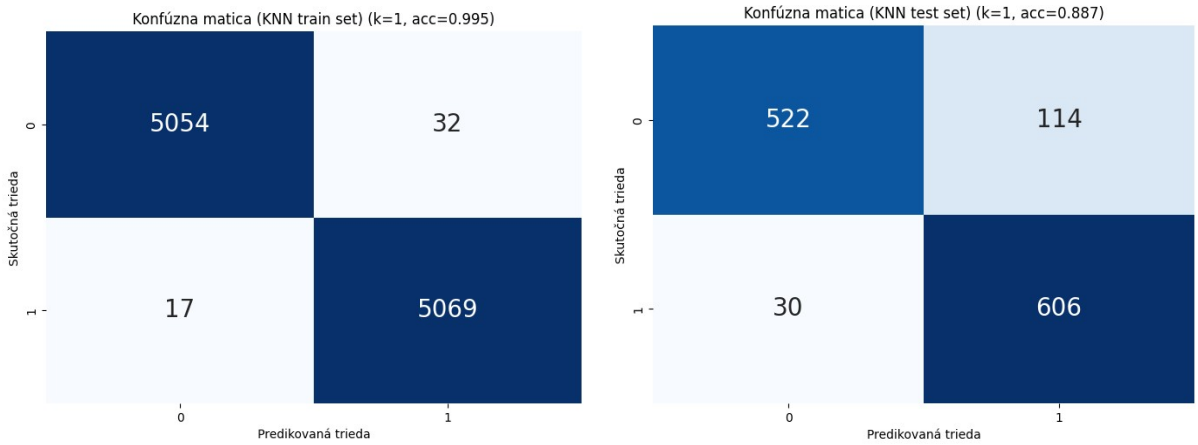
Vidíme, že rozdelenie dát 8:1:1 s 88.68% vyhralo nad všetkým ostatným na základe metriky test úspešnosti, takže je najlepšie natrénovaný model.

1.8 konfúzna matica pre najlepšie natrénovaný KNN model

– Pre najlepšie natrénovaný model vykreslite konfúznu maticu pre trénovaciu aj testovaciu množinu.**0.5b**

Na zobrazovanie konfúznych matíc slúži funkcia vlastná tomuto programu **conf(y_true, y_pred, capa, capb, epochk, acc)**, pričom do y_true putuje realita o y, y_pred predikcia y, capa je typ modelu KNN/NN a na akom type dát train/test, capb či sa tu „epocha“ nazýva epocha či k, epochk číselne výhodná epocha/k a acc presnosť, ktorá je vizualizovaná na konfúznej matici.

Do konfúznej matice odošleme relevantné informácie o **najlepšom 8:1:1 trénovaní KNN**.



konfúzna matica pre 8:1:1 KNN model pre trénovaciu (vľavo) a testovaciu (vpravo) množinu

2.1 EDA

- Analyzujte dataset cez EDA. Pracujte s upraveným aj pôvodným datasetom, aby ste mali k dispozícii všetky údaje (pracujte s dátami po odstránení outlierov a chýbajúcich hodnôt, ale použite hodnoty pred kódovaním, aby ste vedeli použiť slovné hodnoty). Niektoré vzťahy sa Vám budú hľadať lepšie predtým, než dáta upravíte. Nájdite aspoň 5 zaujímavých vzťahov v dátach (každý za 1b), vizualizujte ich pomocou grafov (nie 5x rovnaký), inšpirujte sa ukážkami z iných projektov **(spolu 5b)**:

V prvom rade je dôležité si uvedomiť, že máme tri typy stĺpcov **numerické**, **kategorické**, **binárne**, podľa kt. predikujeme **binárny cieľ**. V **numerickom** sú **si príbuzné označenia príbuzné svojou akosťou** (napr. vek) a dá sa vzťahovať záporná, žiadna či kladná korelácia na numerický cieľ či binárny cieľ (opatrne ak nie je bin.N/Y, tak sme upovedomení len o abs. h. korelácie, ak je bin.N/Y, tak brať ako numerický cieľ), čo nám oznámi, že sa niekde nachádza zaujímavá napr. priama úmera a z toho je možno odvodiť, že je to napr. výkonný parameter. V **binárnom** máme 2 protipólovo odlišné hodnoty, takže dvojtá príbuznosť ostáva a aj symetricky môžeme očakávať vlastnosti ako keď porovnávame numerický stĺpec s binárnym. V **kategorickom** stĺpci nie je dvojtá príbuznosť, ale je ju možno vnútiť tak, že do korelačnej matice vnútime onehot encoded verzie. Toto ale súvisí najmä s multivariátnou analýzou, ktorá je hlavným predmetom 2. časti zadania a EDA pozostáva aj z iných častí.

Program spúšťajme so zapnutými edamap()+cullnull(2000)+culloutl(0.25, 0.75)±dummify(): bivariátna mapa ho potrebuje, univariátna mapa a obe analýzy bežia len bez neho

Negrafická analýza

```
>>> df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
age	8408.0	39.293054	9.671972	18.000	32.000	37.000	46.000	69.000
housing	8408.0	0.544600	0.498036	0.000	0.000	1.000	1.000	1.000
loan	8408.0	0.153782	0.360761	0.000	0.000	0.000	0.000	1.000
contact	8408.0	0.356922	0.479120	0.000	0.000	0.000	1.000	1.000
duration	8408.0	270.197312	207.289728	-252.000	115.000	202.000	374.250	892.000
campaign	8408.0	2.064225	1.293464	1.000	1.000	2.000	3.000	6.000
pdays	8408.0	999.000000	0.000000	999.000	999.000	999.000	999.000	999.000
previous	8408.0	0.000000	0.000000	0.000	0.000	0.000	0.000	0.000
emp.var.rate	8408.0	0.111073	1.566157	-3.400	-1.800	1.100	1.400	1.400
cons.price.idx	8408.0	93.573552	0.568225	92.201	93.075	93.444	93.994	94.767
cons.conf.idx	8408.0	-40.563095	4.492278	-50.800	-42.700	-41.800	-36.400	-29.800
nr.employed	8408.0	5169.426154	70.257916	4963.600	5099.100	5191.000	5228.100	5228.100
subscribed	8408.0	0.243815	0.429408	0.000	0.000	0.000	0.000	1.000
admin.	8408.0	0.276879	0.447483	0.000	0.000	0.000	1.000	1.000
blue-collar	8408.0	0.216461	0.411856	0.000	0.000	0.000	0.000	1.000
entrepreneur	8408.0	0.034848	0.183405	0.000	0.000	0.000	0.000	1.000
housemaid	8408.0	0.024025	0.153135	0.000	0.000	0.000	0.000	1.000
management	8408.0	0.074096	0.261943	0.000	0.000	0.000	0.000	1.000
retired	8408.0	0.030685	0.172473	0.000	0.000	0.000	0.000	1.000
self-employed	8408.0	0.036156	0.186689	0.000	0.000	0.000	0.000	1.000
services	8408.0	0.089795	0.285905	0.000	0.000	0.000	0.000	1.000
student	8408.0	0.021646	0.145534	0.000	0.000	0.000	0.000	1.000
technician	8408.0	0.170909	0.376452	0.000	0.000	0.000	0.000	1.000
unemployed	8408.0	0.024500	0.154606	0.000	0.000	0.000	0.000	1.000
divorced	8408.0	0.110490	0.313518	0.000	0.000	0.000	0.000	1.000
married	8408.0	0.593958	0.491122	0.000	0.000	1.000	1.000	1.000
single	8408.0	0.295552	0.456318	0.000	0.000	0.000	1.000	1.000
basic.4y	8408.0	0.101332	0.301786	0.000	0.000	0.000	0.000	1.000
basic.6y	8408.0	0.052450	0.222946	0.000	0.000	0.000	0.000	1.000
basic.9y	8408.0	0.143673	0.350778	0.000	0.000	0.000	0.000	1.000
high.school	8408.0	0.235014	0.424033	0.000	0.000	0.000	0.000	1.000
illiterate	8408.0	0.000357	0.018887	0.000	0.000	0.000	0.000	1.000
professional.course	8408.0	0.137012	0.343881	0.000	0.000	0.000	0.000	1.000
university.degree	8408.0	0.330162	0.470299	0.000	0.000	0.000	1.000	1.000
mar	8408.0	0.021051	0.143564	0.000	0.000	0.000	0.000	1.000
apr	8408.0	0.070409	0.255850	0.000	0.000	0.000	0.000	1.000
may	8408.0	0.309943	0.462497	0.000	0.000	0.000	1.000	1.000
jun	8408.0	0.136418	0.343252	0.000	0.000	0.000	0.000	1.000
jul	8408.0	0.175547	0.380457	0.000	0.000	0.000	0.000	1.000
aug	8408.0	0.168768	0.374569	0.000	0.000	0.000	0.000	1.000
sep	8408.0	0.013796	0.116652	0.000	0.000	0.000	0.000	1.000
oct	8408.0	0.008325	0.090868	0.000	0.000	0.000	0.000	1.000
nov	8408.0	0.090509	0.286927	0.000	0.000	0.000	0.000	1.000
dec	8408.0	0.005233	0.072155	0.000	0.000	0.000	0.000	1.000
mon	8408.0	0.210871	0.407951	0.000	0.000	0.000	0.000	1.000
tue	8408.0	0.194458	0.395806	0.000	0.000	0.000	0.000	1.000
wed	8408.0	0.193982	0.395438	0.000	0.000	0.000	0.000	1.000
thu	8408.0	0.213011	0.409460	0.000	0.000	0.000	0.000	1.000
fri	8408.0	0.187678	0.390478	0.000	0.000	0.000	0.000	1.000
nonexistent	8408.0	1.000000	0.000000	1.000	1.000	1.000	1.000	1.000

```
>>>
```

účelom tohto príkazu je v **min** a **max** prezradiť koľko dát napr. outlier manažment ukrojil, v **25%** a **75%** ak je použitý IQR outlier manažment, tak aké boli percentily rozhodujúce o ukrojení, **50%** je percentil 50, **mean** je stredná hodnota, smerodajná odchylka **std** prezrádza ako veľmi sú dáta rozptýlené od **meanu**, **count** prezrádza či majú dáta predpokladaný tvar

Ak je problém so zobrazením, môžu pomôcť tieto príkazy:

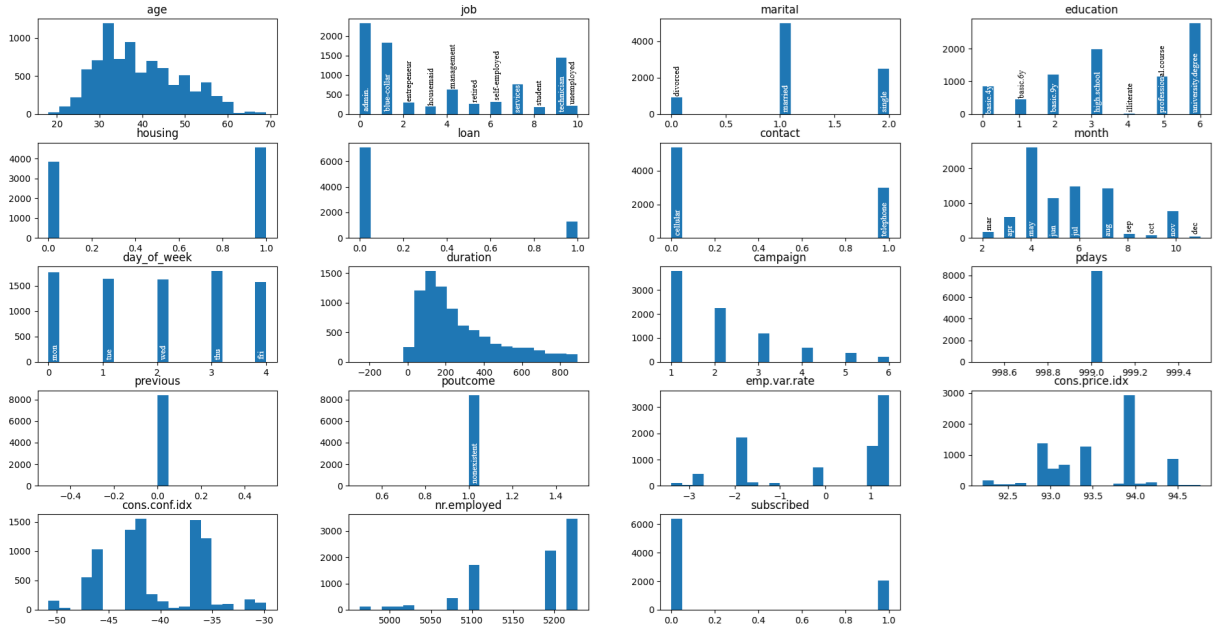
```
>>> pd.set_option('display.max_columns', None)
>>> pd.set_option('display.width', None)
>>>
```

```
>>> df.head().T
                            0       3       5       8       9
age                    60.000  30.000  46.000  43.000  30.000
housing                 1.000   1.000   1.000   1.000   1.000
loan                    0.000   0.000   0.000   0.000   1.000
contact                 1.000   0.000   0.000   1.000   1.000
duration               10.000 402.000 369.000 193.000 131.000
campaign                4.000   1.000   2.000   1.000   2.000
pdays                 999.000 999.000 999.000 999.000 999.000
previous                0.000   0.000   0.000   0.000   0.000
emp.var.rate            1.400  -3.400  -1.800   1.100   1.400
cons.price.idx         94.465  92.649  93.075  93.994  94.465
cons.conf.idx         -41.800 -30.100 -47.100 -36.400 -41.800
nr.employed          5228.100 5017.500 5099.100 5191.000 5228.100
subscribed              0.000   0.000   1.000   0.000   0.000
admin.                  0.000   1.000   0.000   0.000   0.000
blue-collar             0.000   0.000   0.000   1.000   0.000
entrepreneur            0.000   0.000   0.000   0.000   0.000
housemaid               0.000   0.000   0.000   0.000   0.000
management              0.000   0.000   0.000   0.000   0.000
retired                 0.000   0.000   0.000   0.000   0.000
self-employed           1.000   0.000   0.000   0.000   0.000
services                0.000   0.000   0.000   0.000   0.000
student                 0.000   0.000   0.000   0.000   0.000
technician              0.000   0.000   1.000   0.000   1.000
unemployed              0.000   0.000   0.000   0.000   0.000
divorced                0.000   0.000   0.000   0.000   1.000
married                 1.000   0.000   1.000   1.000   0.000
single                  0.000   1.000   0.000   0.000   0.000
basic.4y                0.000   0.000   0.000   1.000   0.000
basic.6y                0.000   0.000   0.000   0.000   0.000
basic.9y                0.000   0.000   0.000   0.000   0.000
high.school             0.000   0.000   0.000   0.000   1.000
illiterate              0.000   0.000   0.000   0.000   0.000
professional.course     1.000   0.000   1.000   0.000   0.000
university.degree       0.000   1.000   0.000   0.000   0.000
mar                     0.000   0.000   0.000   0.000   0.000
apr                     0.000   0.000   1.000   0.000   0.000
may                     0.000   0.000   0.000   1.000   0.000
jun                     1.000   0.000   0.000   0.000   1.000
jul                     0.000   0.000   0.000   0.000   0.000
aug                     0.000   0.000   0.000   0.000   0.000
sep                     0.000   0.000   0.000   0.000   0.000
oct                     0.000   0.000   0.000   0.000   0.000
nov                     0.000   1.000   0.000   0.000   0.000
dec                     0.000   0.000   0.000   0.000   0.000
mon                     0.000   0.000   0.000   0.000   0.000
tue                     0.000   1.000   0.000   1.000   1.000
wed                     0.000   0.000   0.000   0.000   0.000
thu                     0.000   0.000   1.000   0.000   0.000
fri                     1.000   0.000   0.000   0.000   0.000
nonexistent             1.000   1.000   1.000   1.000   1.000
>>>
```

```
>>> df.info()
<class 'pandas.core.frame.DataFrame'>
Index: 8408 entries, 0 to 13137
Data columns (total 50 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   age                  8408 non-null   float64
 1   housing              8408 non-null   float64
 2   loan                 8408 non-null   float64
 3   contact              8408 non-null   float64
 4   duration             8408 non-null   float64
 5   campaign             8408 non-null   float64
 6   pdays                8408 non-null   float64
 7   previous             8408 non-null   float64
 8   emp.var.rate         8408 non-null   float64
 9   cons.price.idx       8408 non-null   float64
 10  cons.conf.idx        8408 non-null   float64
 11  nr.employed          8408 non-null   float64
 12  subscribed           8408 non-null   float64
 13  admin.               8408 non-null   float64
 14  blue-collar          8408 non-null   float64
 15  entrepreneur         8408 non-null   float64
 16  housemaid            8408 non-null   float64
 17  management           8408 non-null   float64
 18  retired              8408 non-null   float64
 19  self-employed        8408 non-null   float64
 20  services             8408 non-null   float64
 21  student              8408 non-null   float64
 22  technician           8408 non-null   float64
 23  unemployed           8408 non-null   float64
 24  divorced             8408 non-null   float64
 25  married              8408 non-null   float64
 26  single               8408 non-null   float64
 27  basic.4y             8408 non-null   float64
 28  basic.6y             8408 non-null   float64
 29  basic.9y             8408 non-null   float64
 30  high.school          8408 non-null   float64
 31  illiterate           8408 non-null   float64
 32  professional.course  8408 non-null   float64
 33  university.degree    8408 non-null   float64
 34  mar                  8408 non-null   float64
 35  apr                  8408 non-null   float64
 36  may                  8408 non-null   float64
 37  jun                  8408 non-null   float64
 38  jul                  8408 non-null   float64
 39  aug                  8408 non-null   float64
 40  sep                  8408 non-null   float64
 41  oct                  8408 non-null   float64
 42  nov                  8408 non-null   float64
 43  dec                  8408 non-null   float64
 44  mon                  8408 non-null   float64
 45  tue                  8408 non-null   float64
 46  wed                  8408 non-null   float64
 47  thu                  8408 non-null   float64
 48  fri                  8408 non-null   float64
 49  nonexistent          8408 non-null   float64
dtypes: float64(50)
memory usage: 3.3 MB
>>>
```

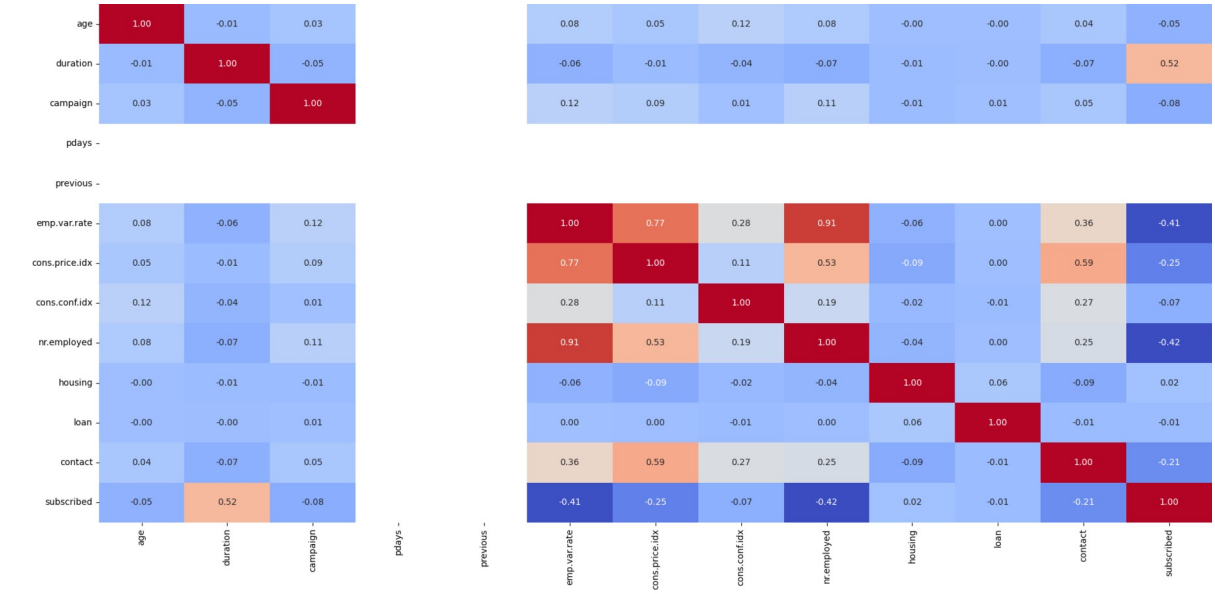
vľavo vidíme náhľad do prvých 5 dát datasetu po cleaningu, vpravo vidíme debugovacie info, dáta prezrádzajú, že neboli zle castnuté a dummies fungujú cast dát je np.float64

Univariátna mapa



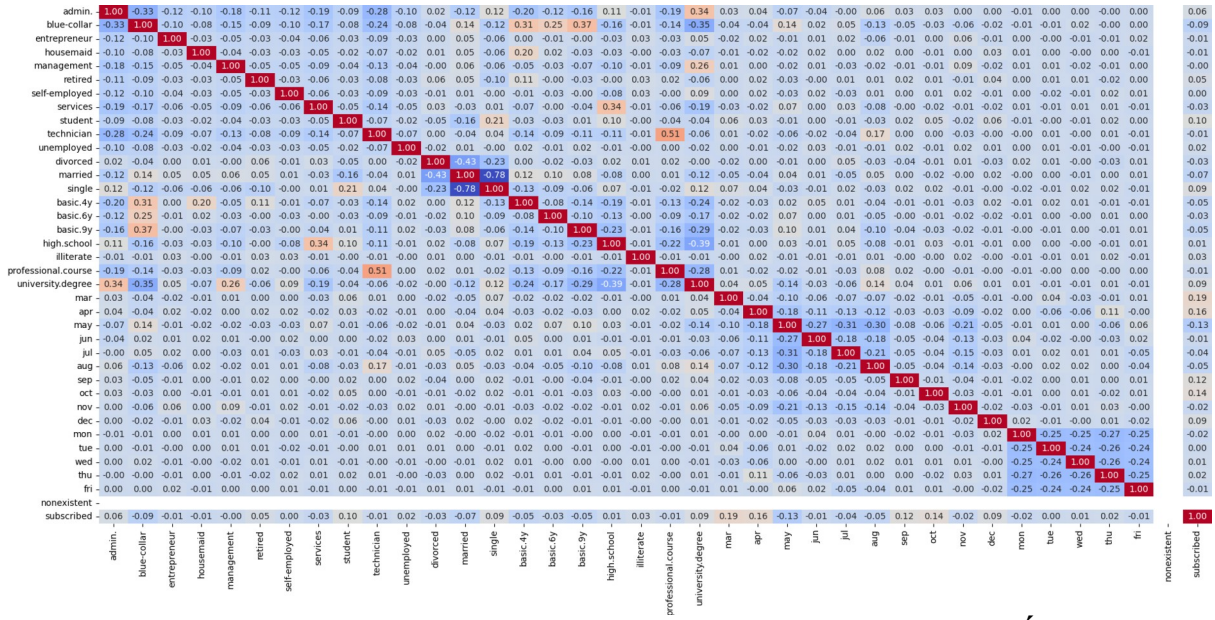
na grafoch univariátnej analýzy vidíme, **ktoré typy stĺpcov sme dobre zdokumentovali** a podľa výsledkov multivariátnej analýzy budeme vedieť či je potreba získať viac dát o tipe na silne korelovanú konverziu priamo či nepriamo s cieľovou premennou

Bivariátna mapa num-num



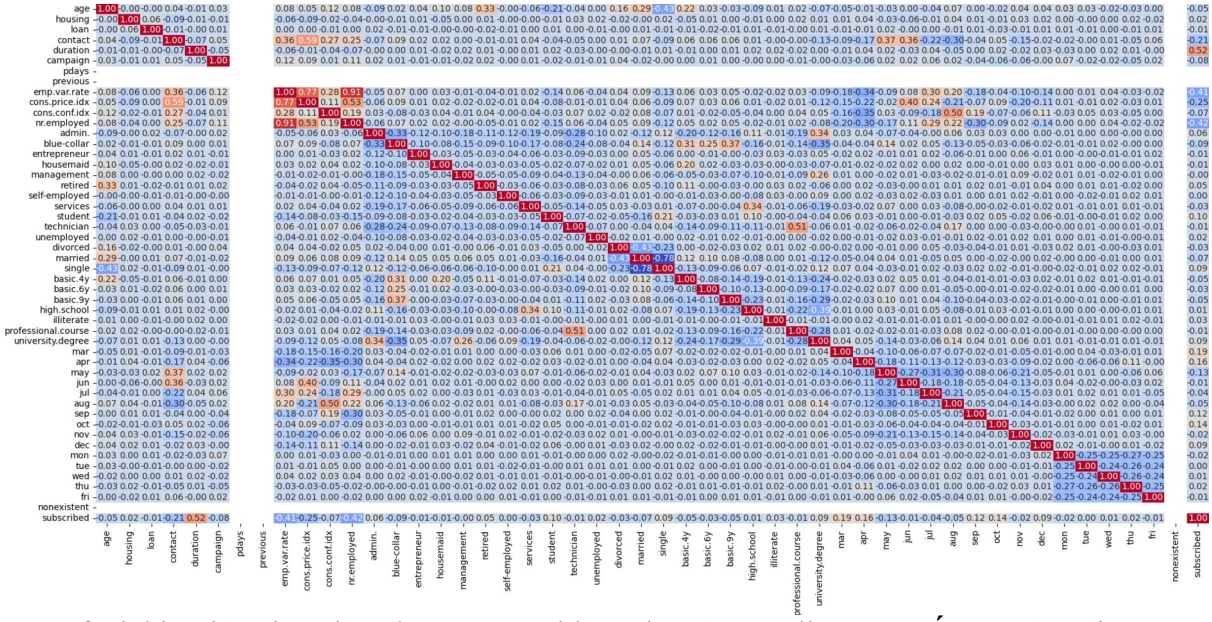
na grafoch bivariátnej analýzy num-num vidíme, ktoré typy **numerických** a **binárnych stĺpcov** sú vzájomne záporne, nulovo či kladne korelované, toto má slúžiť ako **mapa na zostrojenie multivariátnych vzťahov**, ktoré sú ešte preverené grafom proti napr. cieľovej premennej alebo niečomu, čo k nej nepriamo vedie

Bivariátna mapa kateg-kateg



na grafoch bivariátnej analýzy kateg-kateg vidíme, ktoré typy **kategorických stĺpcov** sú vzájomne záporne, nulovo či kladne korelované, toto má slúžiť ako **mapa na zostrojenie multivariátnych vzťahov**, ktoré sú ešte preverené grafom proti napr. cieľovej premennej alebo niečomu, čo k nej nepriamo vedie

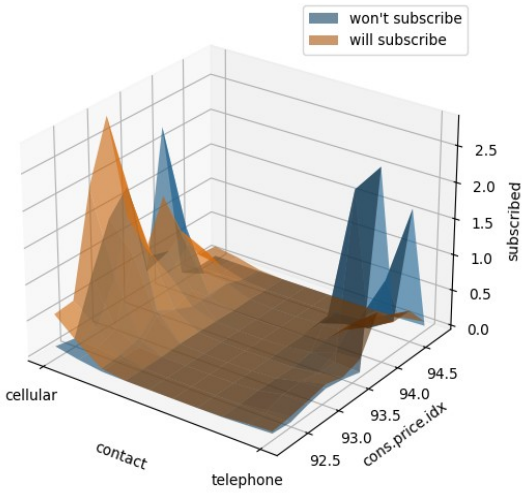
Bivariátna mapa kateg-num



na grafoch bivariátnej analýzy kateg-num vidíme, ktoré typy **všetkých stĺpcov** sú vzájomne záporne, nulovo či kladne korelované, toto má slúžiť ako **mapa na zostrojenie multivariátnych vzťahov**, ktoré sú ešte preverené grafom proti napr. cieľovej premennej alebo niečomu, čo k nej nepriamo vedie

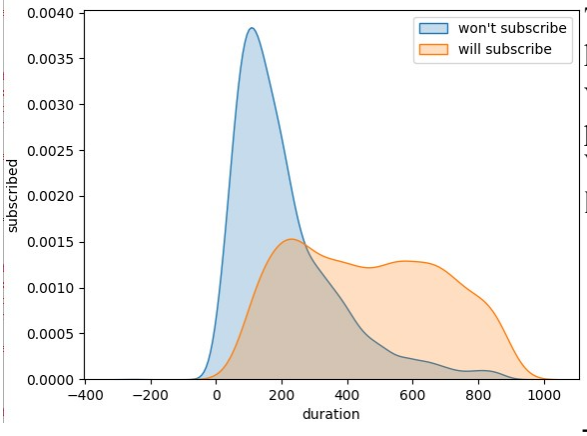
Multivariátne náleziská

a) subscribed<->contact(-.21)<->cons.price.idx(.59)



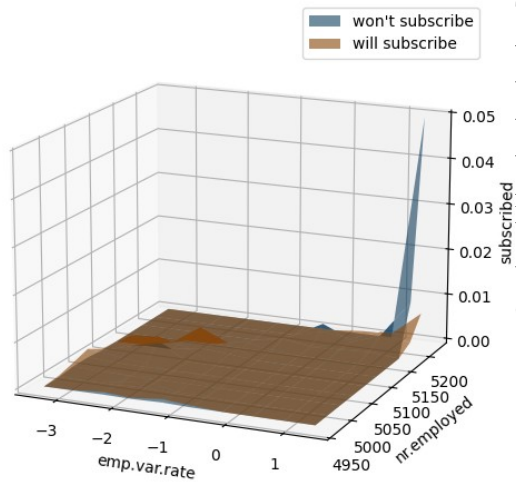
Tento vzťah značí to, že volanie pevnou linkou (contact=1) je nepriamo úmerné s konverziou <=> volanie mobilným telefónom je priamo úmerné s konverziou, čo vidno na zdvihnutej hustote **will subscribe** na strane cellular oproti strane telephone a je možné zvýrazniť konverziu pri volaní mobilom, ak prebehne v čase keď je mesačný ukazovateľ cons.price.idx nízky.
Teda vysoká konverzia, keď je volanie mobilom a nízky cons.price.idx.

b) subscribed<->duration(.52)



Tento vzťah značí to, že dĺžka posledného kontaktu je priamo úmerná s konverziou, čo vidno na zdvihnutej hustote **will subscribe** na pravej strane.
Vysoká konverzia, ak bola dĺžka posledného kontaktu vysoká.

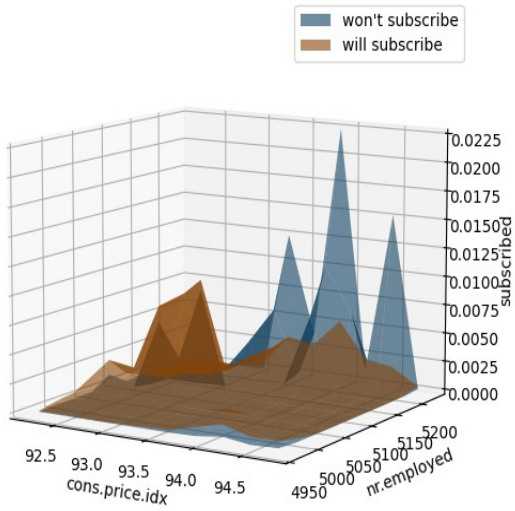
c) subscribed<->emp.var.rate(-.41)<->nr.employed(.91)



Tento vzťah značí to, že emp.var.rate je nepriamo úmerný s konverziou a je možné ešte pridať výraznému vzťahu záporného javu **won't subscribe** tým, že by sa nielen hovorilo so zákazníkom za vysokého emp.var.rate, ale ešte aj za vysokého nr.employed.

Veľkému peaku nekonverzie vpravo vzadu sa vyhneme keď so zákazníkom hovoríme za nízkeho emp.var.rate aj nr.employed.

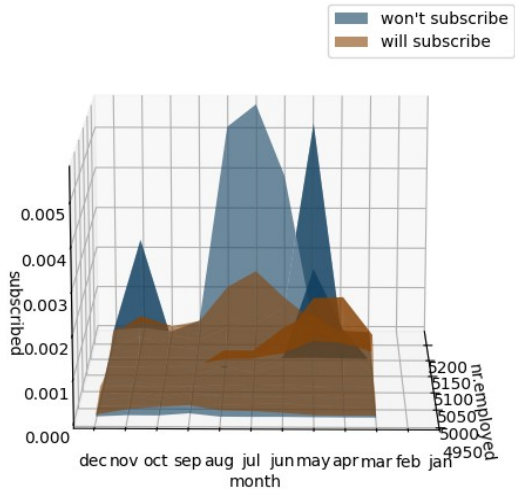
d) subscribed<->cons.price.idx(-.25)<->nr.employed(.53)



Tento vzťah značí to, že cons.price.idx je nepriamo úmerný s konverziou a je možné ešte pridať výraznému vzťahu záporného javu **won't subscribe** tým, že by sa nielen hovorilo so zákazníkom za vysokého cons.price.idx, ale aj za vysokého nr.employed.

Veľkému peaku nekonverzie jemne vpravo vzadu sa vyhneme keď so zákazníkom hovoríme za nízkeho cons.price.idx aj nr.employed.

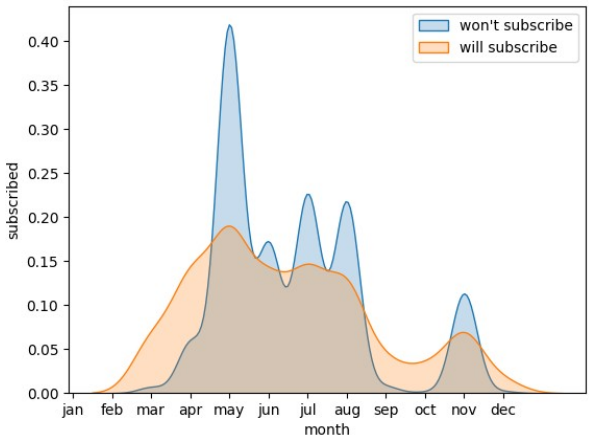
e)subscribed<->nr.employed(-.42)<->jul(.29)



Tento vzťah značí to, že nr.employed je nepriamo úmerný s konverziou a je možné ešte pridať výraznému vzťahu záporného javu **won't subscribe** tým, že by sa nielen hovorilo so zákazníkom za vysokého nr.employed, ale aj v júli.

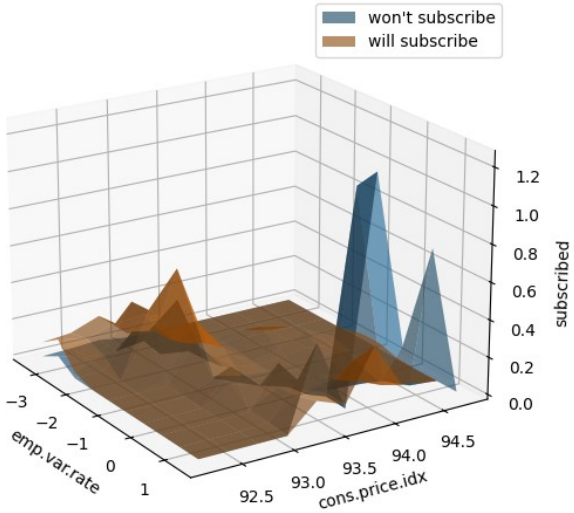
Veľkému peaku nekonverzie v strede vzadu sa vyhneme keď so zákazníkom hovoríme za nízkeho nr.employed a hlavne nie v júli.

f) subscribed<->mar(.19)



Tento vzťah značí to, že v rôzne mesiace je rozumnejšie alebo menej rozumné hovoriť so zákazníkom. Marec sa javí kladne v tom, že zákazníci disproporčne volia **will subscribe** oproti **won't subscribe** podľa pomeru will subscribe:won't subscribe hustôt, hoci hustota prezrádza, že možno chýba viac dát, lebo iné mesiace sú lepšie dokumentované a potvrdzuje to aj univariátna analýza stĺpca month. **Vysoký pomer konverzie:nekonverzii, ak v marci.**

g) subscribed<->emp.var.rate(-.41)<->cons.price.idx(.77)

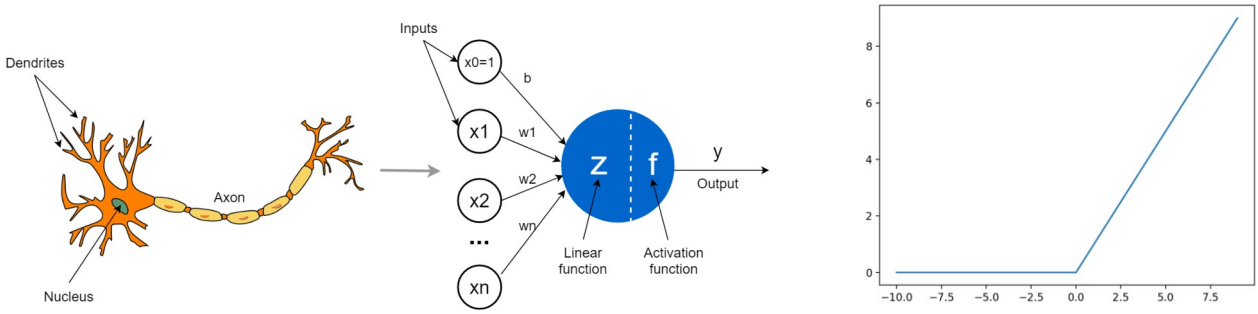


Tento vzťah značí to, že emp.var.rate je nepriamo úmerný s konverziou a je možné ešte pridať výraznému vzťahu záporného javu **won't subscribe** tým, že by sa nielen hovorilo so zákazníkom za vysokého emp.var.rate, ale aj za vysokého cons.price.idx. **Veľkému peaku nekonverzie vpravo sa vyhneme keď so zákazníkom hovoríme za nízkeho emp.var.rate aj nízkeho cons.price.idx.**

3.1 plnohodnotná neurónová sieť

– Zvoľte architektúru a nastavenia hyperparametrov tak, aby ste dosiahli pretrénovanie. Demonštrujte pomocou grafov priebehu trénovania, vyhodnotenia úspešností a konfúznej matice pre trénovaciu aj testovaciu množinu. **1b**

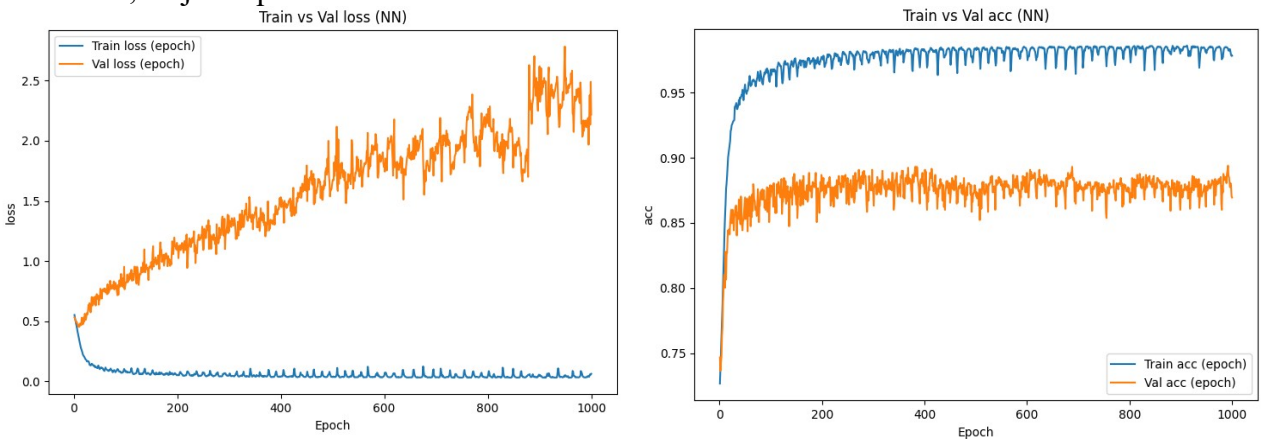
Podstata neurónovej siete spočíva v tom, že na vstupy sú navedené evalvované číselné hodnoty (**vstupná vrstva**), po čom tieto sa všetky zvezú na každý neurón ďalšej vrsty (**1. skrytá vrstva**) cez **váhy**, ktoré majú rôznu hodnotu, ktorou prenásobia cestujúce číslo a takto má jeden neurón teoretickú multiplikatívnu kontrolu nad tým, akú majú všetky neuróny predchádzajúcej skrytej vrstvy či vstupnej vrstvy relevanciu pri aditívnom príchode doň. Po sčítaní prenásobeného pridáme **bias**, ktorý je v sieti informáciou ako dôležitý je neurón bezohľadu na to, že doňho ústia veľmi nízke váhy. V poslednej fáze sčítané prenásobené s pridaným biasom necháme prejsť **aktivačnou funkciou** (často ReLU) a toto putuje cyklicky naprieč ľubovoľným počtom skrytých vrstiev až po **výstupnú vrstvu**, ktorá sa nachádza za **poslednou skrytou vrstvou. Výstupná vrstva** je výsledkom evalvácie číselných hodnôt zo **vstupnej vrstvy.**



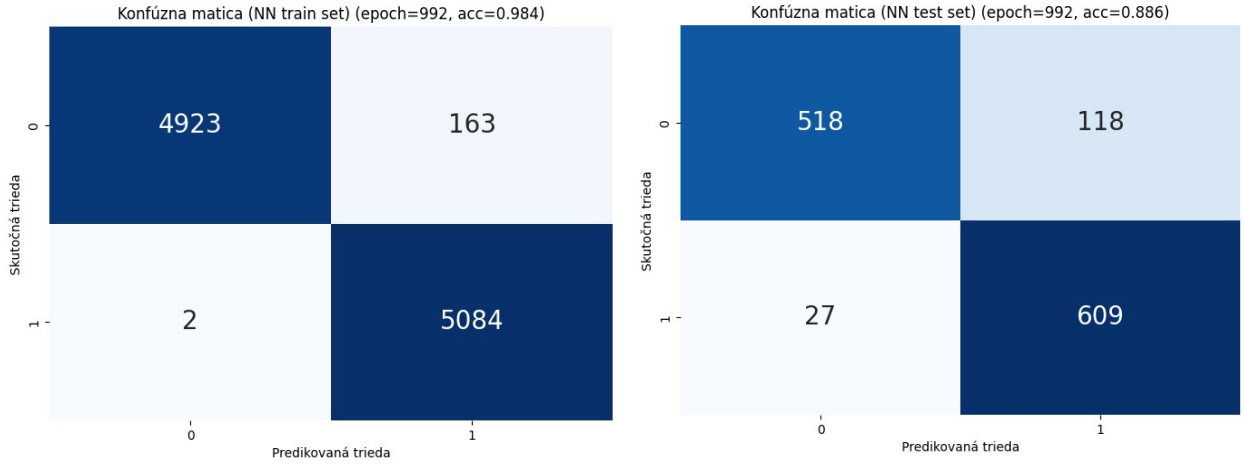
y=ReLU(x)

Charakteristika

Sieť použitú v kóde zadania môžeme charakterizovať tak, že má dataset delený 8:1:1, má 48 vstupných neurónov, 128 v 1. skrytej vrstve, 64 v 2. skrytej vrstve a 2 neuróny vo výstupnej vrstve. Všetky aktivačné funkcie sú ReLU a sú na všetkých pozíciach a biasy sú povolené všade. Používali sme learning rate 0.005, čo symbolizuje zdržanlivosť v menení hyperparametrov váh a biasov oproti vyššiemu learning rate. Bola zvolená veľkosť batchov 64. Používa sa loss funkcia CrossEntropyLoss a optimizer Adam. Ak nie je uvedené inak, max. počet epochs je 1K, patience 30, ktorý súvisí s EarlyStopping. V sieti bolo dosiahnuté pretrénovanie, ktoré spočíva v tom, že sieť namiesto zovšeobecnenia na nevideného klienta voľnejšou krivkou začne memorovať trénovaciu množinu dát napnutou krivkou, čo je nesprávnou evolúciou.



priebeh trénovania pre NN model pre trénovaciu (vľavo) a testovaciu (vpravo) množinu 3.1 charakteristika (ale **patience>>**), val. presnosť už nerastie <=> pretrénovanie



konfúzna matica pre NN model pre trénovaciu (vľavo) a testovaciu (vpravo) množinu

```
Epoch 990 | train_loss=0.0344 | train_acc=0.985 | val_loss=2.1156 | val_acc=0.888
Epoch 991 | train_loss=0.0375 | train_acc=0.984 | val_loss=2.1135 | val_acc=0.890
Epoch 992 | train_loss=0.0365 | train_acc=0.983 | val_loss=2.1404 | val_acc=0.894
Epoch 993 | train_loss=0.0444 | train_acc=0.982 | val_loss=2.1780 | val_acc=0.886
Epoch 994 | train_loss=0.0442 | train_acc=0.982 | val_loss=2.1900 | val_acc=0.885
Epoch 995 | train_loss=0.0434 | train_acc=0.983 | val_loss=1.9678 | val_acc=0.881
Epoch 996 | train_loss=0.0393 | train_acc=0.983 | val_loss=2.1524 | val_acc=0.877
Epoch 997 | train_loss=0.0537 | train_acc=0.981 | val_loss=2.3801 | val_acc=0.881
Epoch 998 | train_loss=0.0607 | train_acc=0.979 | val_loss=2.1341 | val_acc=0.878
Epoch 999 | train_loss=0.0567 | train_acc=0.978 | val_loss=2.4903 | val_acc=0.870
Epoch 1000 | train_loss=0.0637 | train_acc=0.978 | val_loss=2.2186 | val_acc=0.869
Najlepšia epocha=992
Train accuracy: 98.38%
Test accuracy: 88.60%
OK
>>>
```

validácie->skoré zastavenie->oznámenie výhodnej epochy a trénovacej a testovacej presnosti Konštatujeme, že trénovacia presnosť dosahuje 98.38% a testovacia presnosť dosahuje 88.60%.

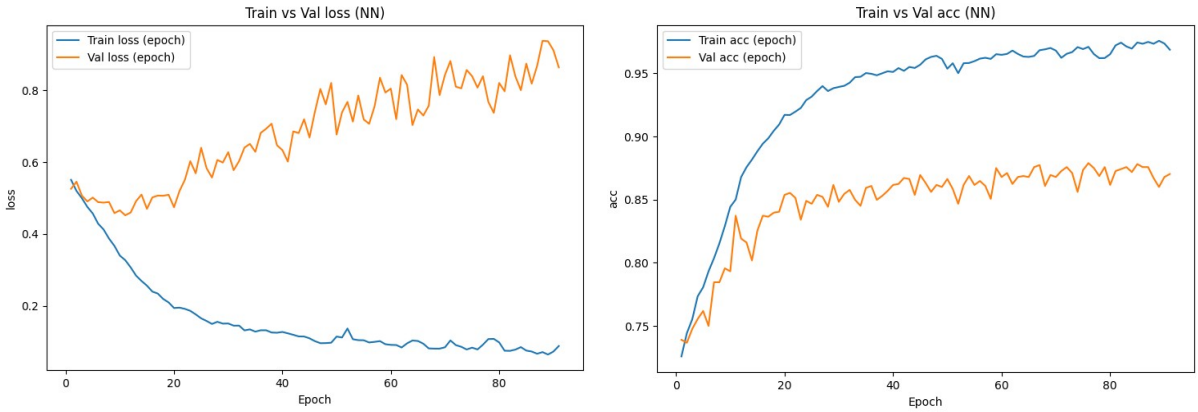
3.2 EarlyStopping v NN

– Odstráňte pretrénovanie tak, že do trénovania zavediete EarlyStopping pre skoré zastavenie trénovania. Demonštrujte pomocou grafov priebehu trénovania, vyhodnotenia úspešností a konfúznej matice pre trénovaciu aj testovaciu množinu

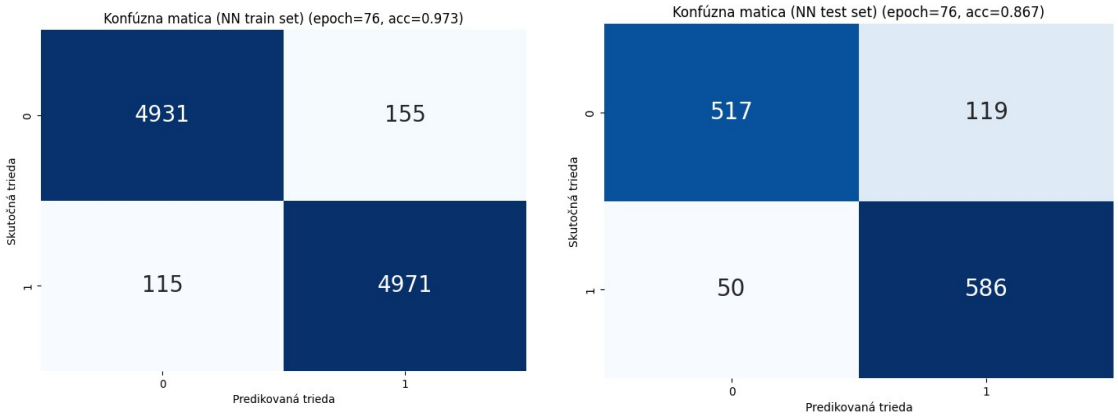
1b

EarlyStopping spočíva v snahe zastaviť trénovanie, keď sieť začína byť zjavne pretrénovaná a ďalšie užitočné trénovanie za účelom lepšieho produkčného spracovávania nevidených dát už aj tak neprebehne. Ako metrika toho, že sieť sa už nezlepší je použitá **stagnujúca najlepšia validačná presnosť**. Ak sa nezlepší patience-krát epoch, tak program skončí skôr než prejde maximálny počet epoch epochs.

```
if epoch_val_acc > best_val_acc:
    best_val_acc = epoch_val_acc
    best_state = copy.deepcopy(model.state_dict())
    wait = 0
    best_epoch = epoch
else:
    wait += 1
    if wait >= patience:
        stopped_epoch = epoch
        print(f'Early stopping po {stopped_epoch} epochách (patience={patience}). '
              f'Najlepšia val_acc={best_val_acc:.4f}')
        break
```



priebeh trénovania pre NN model pre trénovaciu (vľavo) a testovaciu (vpravo) množinu
3.1 charakteristika (ale **patience=15**), ak porovnáme s 3.1 (patience>>) zjavne nestagnuje



konfúzna matica pre NN model pre trénovaciu (vľavo) a testovaciu (vpravo) množinu

```
Epoch 80 | train_loss=0.0979 | train_acc=0.965 | val_loss=0.8206 | val_acc=0.862
Epoch 81 | train_loss=0.0748 | train_acc=0.972 | val_loss=0.7973 | val_acc=0.873
Epoch 82 | train_loss=0.0744 | train_acc=0.974 | val_loss=0.8975 | val_acc=0.874
Epoch 83 | train_loss=0.0780 | train_acc=0.971 | val_loss=0.8383 | val_acc=0.876
Epoch 84 | train_loss=0.0848 | train_acc=0.970 | val_loss=0.8004 | val_acc=0.872
Epoch 85 | train_loss=0.0752 | train_acc=0.974 | val_loss=0.8743 | val_acc=0.878
Epoch 86 | train_loss=0.0725 | train_acc=0.973 | val_loss=0.8186 | val_acc=0.876
Epoch 87 | train_loss=0.0664 | train_acc=0.975 | val_loss=0.8682 | val_acc=0.876
Epoch 88 | train_loss=0.0709 | train_acc=0.973 | val_loss=0.9379 | val_acc=0.867
Epoch 89 | train_loss=0.0642 | train_acc=0.976 | val_loss=0.9368 | val_acc=0.860
Epoch 90 | train_loss=0.0724 | train_acc=0.974 | val_loss=0.9119 | val_acc=0.868
Epoch 91 | train_loss=0.0879 | train_acc=0.969 | val_loss=0.8639 | val_acc=0.870
Early stopping po 91 epochách (patience=15). Najlepšia val_acc=0.8789
Najlepšia epocha=76
Train accuracy: 97.35%
Test accuracy: 86.71%
OK
>>>
```

validácie->skoré zastavenie->oznámenie výhodnej epochy a trénovacej a testovacej presnosti
Konštatujeme, že trénovacia presnosť dosahuje 97.35% a testovacia presnosť dosahuje 86.71%.

3.3, 3.4 predvádzanie NN modelu s rôznym lr či architektúrou siete

* Jednotlivé konfigurácie sú prehľadne zapísané v JEDNEJ tabuľke (stačí zapísať rozdiely v experimentoch, netreba pre každý experiment uvádzať všetky parametre, ak ostali zachované). **1b**
* Pre všetky trénovania sú v tabuľke vyhodnotené dosiahnuté úspešnosti pre trénovaciu aj testovaciu množinu. **1b**

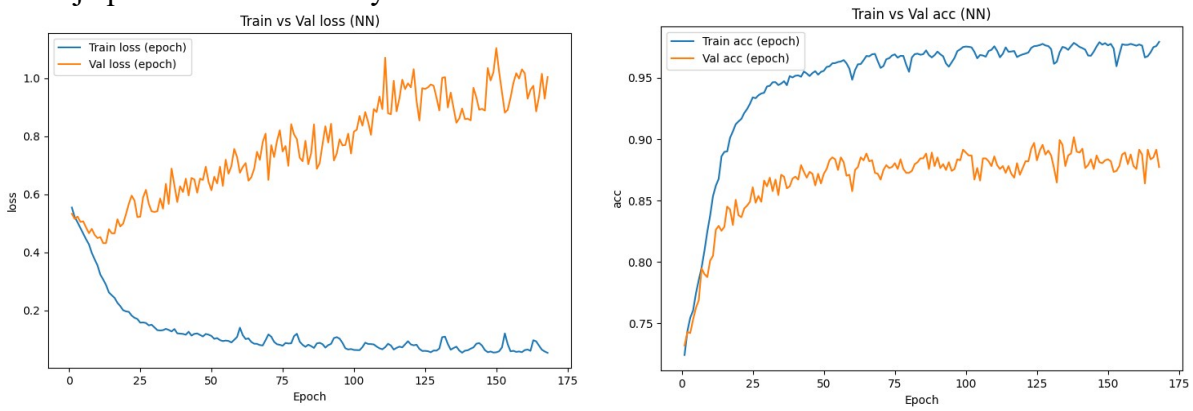
Konfigurácia s výnimkou častí, na ktoré je upozornené v tabuľke je taká ako v 3.1 charakteristike (**patience=30**). Šedou je vyznačené bežné nastavenie.

learning rate	počet neurónov	train úspešnosť	test úspešnosť
0.05	**48->128->64->2**	**76.98%**	**72.80%-prehráva**
0.01	48->128->64->2	93.90%	86.56%
0.005	48->128->64->2	97.36%	88.21%
0.001	48->128->64->2	97.86%	88.05%
0.005	**48->256->128->2**	**97.86%**	**89.62%-vyhráva**
0.005	48->64->32->2	94.48%	84.59%

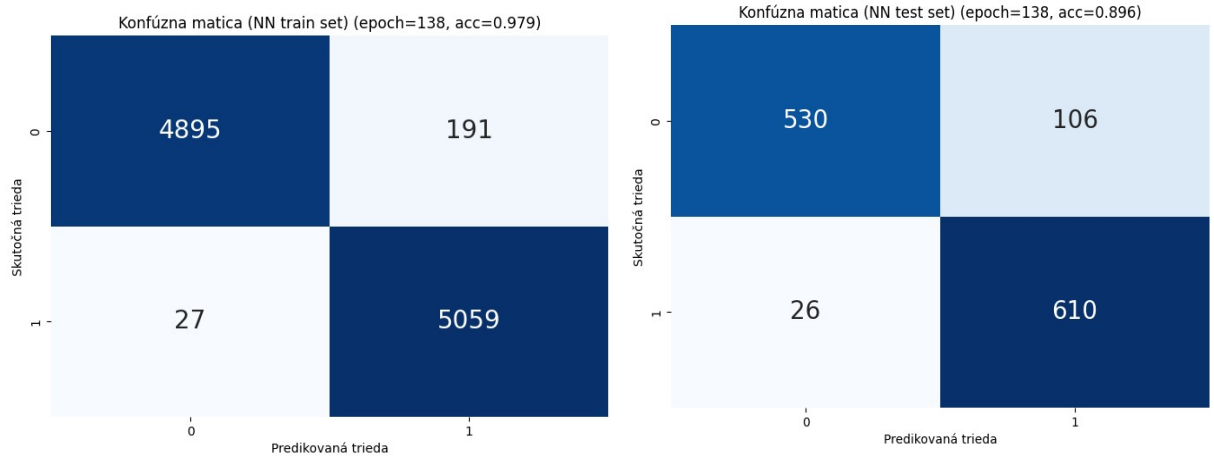
3.5 štatistiky k najlepšiemu a najhoršiemu pokusu v predvádzaní NN modelu

* Pre najlepšie a najhoršie trénovanie je navyše zobrazený priebeh trénovania a konfúzne matice pre trénovaciu aj testovaciu množinu. **1b**

Pre najlepšie lr=0.001 a vrstvy: 48->128->64->2 s 89.62 % test:

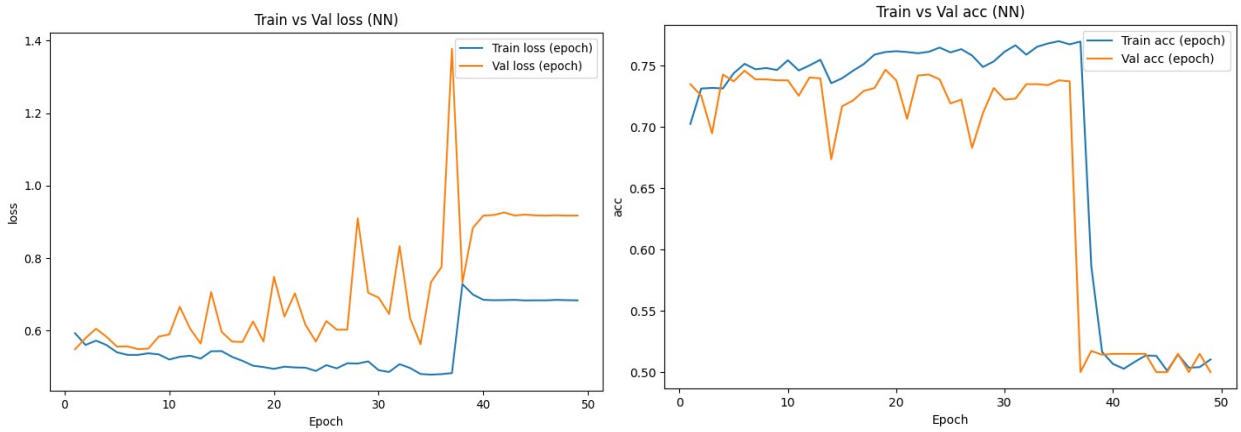


priebeh trénovania pre NN model pre trénovaciu (vľavo) a testovaciu (vpravo) množinu

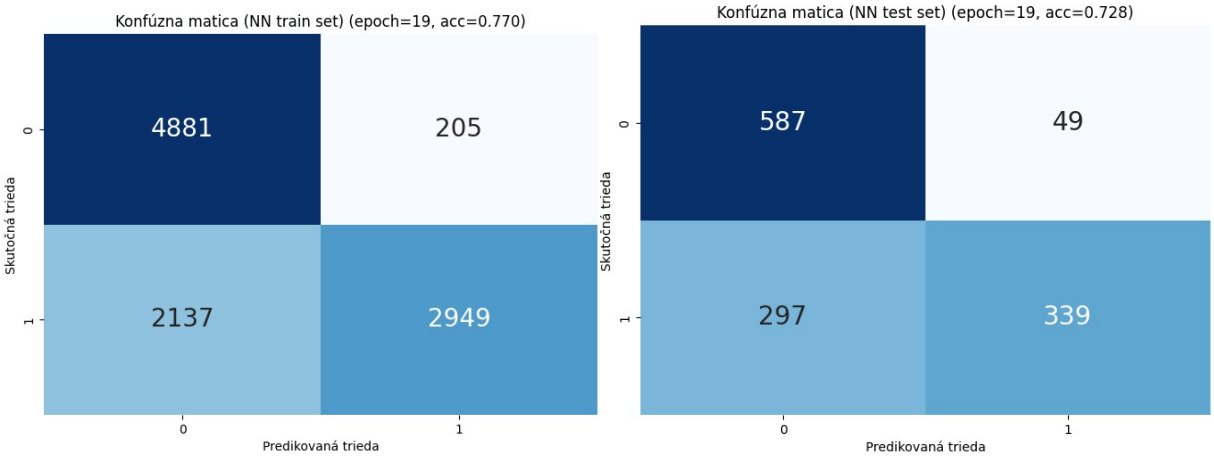


konfúzna matica pre NN model pre trénovaciu (vľavo) a testovaciu (vpravo) množinu

Pre najhoršie lr=0.05 a vrstvy: 48->128->64->2 s 72.80% test:



priebeh trénovania pre NN model pre trénovaciu (vľavo) a testovaciu (vpravo) množinu



konfúzna matica pre NN model pre trénovaciu (vľavo) a testovaciu (vpravo) množinu

Záver

V práci bol zapracovaný kód **cvicenie_1.py** a **cvicenie_2.py**, ktorý bol prevzatý a adaptovaný, autormi sú učitelia predmetu I-SUNS v 2025/2026 na FEI STU BA. Zdroj na workflow pri EDA je https://medium.com/data-science/semi-automated-exploratory-data-analysis-eda-in-python-7f96042c9809 a https://www.dasca.org/newsroom/a-comprehensive-guide-to-mastering-exploratory-data-analysis (už neexistuje).

Použitie AI

V časti data cleaningu a jednoduchého klasifikátora bolo AI ChatGPT použité len na jednoduché až stredne náročné hľadanie syntaxe a jednoduché až stredne náročné opravy kódu. V časti neurónovej siete bolo AI použité vo väčšej miere na generáciu funkcií **conf** a **priebeh**. V práci bolo použité AI vo väčšom meradle na generáciu a opravu kódu, ktorý by generoval **EDA 3D graf hustoty** a v menšom meradle na generáciu a opravu kódu súvisiaceho s jednoduchšími EDA grafmi na konci kódu.